OOP in PHP — Question Bank

How to use this: Work through one episode at a time. For each question, either write your answer out (for concept questions) or code it up in a `.php` file (for coding challenges). Paste your answer back to me and I'll grade it, point out gaps, and push back if you violate any of the course principles.

Rule: No peeking at the notes while you answer. Recall first, reference second.

Episode 1 — Classes

Q1.1 (Concept)

In your own words, what is the difference between a **class** and an **object**? Give a non-code analogy.

Q1.2 (Concept)

What does the `__construct` method do, and when does it run?

Q1.3 (Code)

Write a `Movie` class with `title`, `director`, and `year` properties using **constructor property promotion**. Add a `summary()` method that returns: `"Inception (2010) — directed by Christopher Nolan"`. Instantiate it and echo the summary.

Q1.4 (Trap)

What's wrong with this code, and how would you fix it using what you learned in Episode 1?

```
function createUser($data) {
    return [
        'name' => $data['name'],
        'email' => $data['email'],
        'role' => $data['role'] ?? 'viewer',
    ];
}
```

Episode 2 — Objects

Q2.1 (Concept)

Explain the difference between `==` and `===` when comparing two objects. Give a scenario where each makes sense.

Q2.2 (Predict the output)

```php
class Box { public int $value = 1; }

$a = new Box();
$b = $a;
$b->value = 99;

$c = new Box();
$c->value = 99;

var_dump($a->value);
var_dump($a === $b);
var_dump($a === $c);
var_dump($a == $c);
```

What does each `var_dump` print? Explain **why**.

Q2.3 (Code)

Write a function `resetBox(Box $box): void` that sets `$box->value` back to `0`. Show with code that calling `resetBox($a)` modifies the original `$a` — and explain why that happens.

Q2.4 (Fix it)

You want `$b` to be an independent copy of `$a` so mutating `$b` doesn't affect `$a`. What keyword do you use?

Episode 3 — DTOs, Types, and Static Analysis

Q3.1 (Concept)

What is a DTO, and why would you use one instead of an associative array for passing data between layers of your app? Give two concrete benefits.

Q3.2 (Code)

Convert this into a proper DTO with all the PHP 8.2+ features you've learned (readonly, enum, typed properties):

```php
$signupData = [
    'email' => 'ben@stratusolve.co.za',
    'password' => 'secret123',
    'plan' => 'pro',   // must be one of: free, pro, enterprise
    'referralCode' => null,
];
```

Q3.3 (Trap)

A junior dev writes this:

```php
readonly class UserDTO {
    public function __construct(
        public string $name,
        public array $preferences = [],
    ) {}
}

$dto = new UserDTO('Ben');
$dto->preferences[] = 'dark-mode';   // does this work?
```

Does it work? Why or why not? What's the right way to model mutable-looking collections on a readonly DTO?

Q3.4 (Concept)

What does PHPStan do that PHP itself doesn't? Name one type of bug it would catch that your app wouldn't notice until runtime.

Episode 4 — Dependencies, Coupling, and Interfaces

Q4.1 (Concept)

What does "new is glue" mean? Give an example of a class that's hard to test *because* of this problem.

Q4.2 (Spot the hard dependency)

What's wrong with this class from a coupling perspective? Rewrite it properly.

```
class ReportGenerator
{
    public function generate(): string
    {
        $db = new MySqlDatabase('localhost', 'root', 'password');
        $rows = $db->query('SELECT * FROM sales');
        return json_encode($rows);
    }
}
```

Q4.3 (Code)

Design the interface and two implementations for a logger: one writes to a file, the other writes to stdout. Then write a `PaymentProcessor` class that accepts any logger and logs a message when a payment succeeds.

Q4.4 (Concept)

Why should the type hint in a constructor be an **interface** rather than a **concrete class**? What do you lose if you type-hint the concrete class?

Episode 5 — Inheritance and Abstract Classes

Q5.1 (Concept)

What's the difference between an **abstract class** and an **interface**? When would you reach for one over the other?

Q5.2 (Code)

Build an abstract class `Shape` with: - an abstract method `area(): float` - a concrete method `describe(): string` that returns `"This shape has an area of X"` (using the subclass's `area()`)

Then implement `Circle` and `Rectangle` subclasses.

Q5.3 (Push back)

Your teammate wants to model this hierarchy:

```
Vehicle
  └── Car
        └── ElectricCar
              └── TeslaModelS
```

What's wrong with this? What's the course's preferred approach?

Q5.4 (Concept)

What does `parent::method()` do, and when would you use it instead of just calling `method()` directly?

Episode 6 — Interfaces as Feature Filters

Q6.1 (Concept)

Episode 4 framed interfaces as *contracts*. Episode 6 frames them as *capability markers*. Explain the difference with an example.

Q6.2 (Code)

Create a `Discountable` interface with `discountPercent(): int`. Make `Product` and `Subscription` both implement it. Write a function `totalAfterDiscounts(array $items): float` that: - accepts an array of mixed objects (some `Discountable`, some not) - applies the discount only to items that implement `Discountable` - returns the final total

Q6.3 (Concept)

Why is it better to have five small interfaces (`Cacheable`, `Sortable`, `Searchable`, `Exportable`, `Archivable`) than one big `DataObject` interface that declares all those methods?

Q6.4 (Trap)

A class implements `Sortable` but also has a `sortKey()` method that returns a hard-coded `0` because "we'll figure out sorting later." What's wrong with this, even though it technically compiles?

Episode 7 — Encapsulation and Visibility

Q7.1 (Concept)

In one sentence, what is encapsulation *for*? (Hint: it's not about secrecy.)

Q7.2 (Visibility drill)

For each property below, pick `public`, `protected`, `private`, or `public private(set)` and justify: - a `User`'s `id` (set once at creation, read everywhere) - a `ShoppingCart`'s `items` array (mutated only through `addItem()` / `removeItem()`) - a `Logger`'s internal `buffer` (nobody outside the class should see or touch it) - a `Report`'s `title` (fully read/write from anywhere)

Q7.3 (Code)

Write a `Temperature` class that: - accepts a Celsius value in the constructor - exposes `celsius` as publicly readable but not writable from outside - exposes a `fahrenheit` property that's computed on read - throws if initialized below absolute zero (−273.15°C)

Use PHP 8.4 features.

Q7.4 (Push back)

Someone says: "*I make all my properties public because getters and setters are annoying boilerplate.*" What's the right response given what you learned in Ep 7 and Ep 8?

Episode 8 — From Getters and Setters to Property Hooks

Q8.1 (Concept)

Why does a class full of trivial getters like `getName() { return $this->name; }` "leak encapsulation" even though the properties are private?

Q8.2 (Refactor)

Refactor this to use PHP 8.4 property hooks:

```
class Product
{
    private string $name;
    private float $price;

    public function __construct(string $name, float $price) {
        $this->setName($name);
        $this->setPrice($price);
    }

    public function getName(): string { return $this->name; }
    public function setName(string $name): void {
        $this->name = ucwords(strtolower($name));
    }

    public function getPrice(): float { return $this->price; }
    public function setPrice(float $price): void {
        if ($price < 0) throw new InvalidArgumentException('Negative price');
        $this->price = $price;
    }

    public function getFormattedPrice(): string {
```

```
        return 'R' . number_format($this->price, 2);
    }
}
```

Q8.3 (Code)

Write a `User` class with an `email` property that uses a `set` hook to: - lowercase the value - trim whitespace - throw if the format is invalid And a `domain` property (virtual — no storage) that uses a `get` hook to return everything after the `@`.

Q8.4 (Judgment call)

When is a plain `public` property still the right choice, even with property hooks available?

Episode 9 — Object Composition and Abstractions

Q9.1 (Concept)

State the **Dependency Inversion Principle** in your own words. How does it relate to Episode 4's "depend on interfaces"?

Q9.2 (Design)

You need to send notifications to users. Each notification has a **channel** (email, SMS, push) and a **format** (plain text, markdown, HTML). Design this with composition — show the interfaces and how a `NotificationService` would be constructed. Do **not** use inheritance.

Q9.3 (Leaky abstraction)

Someone defines this interface:

```
interface FileStore {
    public function put(string $path, string $data): void;
    public function get(string $path): string;
    public function ftpUpload(string $path, string $data): void;
    public function getS3Url(string $path): string;
}
```

What's leaking? Fix the interface.

Q9.4 (Push back)

Your teammate creates an `AbstractUserRepository` interface because "we might swap the database one day." There's currently only one implementation (`MySqlUserRepository`), no tests that mock it, and no plan to add another. What's the course's take on this?

Episode 10 — OOP Workshop (Capstone)

Q10.1 (Walkthrough)

In the file-storage API from Ep 10, what specific role does the `Storage` interface play? What would break if you type-hinted `LocalStorage` everywhere instead?

Q10.2 (Code — the whole thing)

Without looking at the notes, design and implement: - a `Cache` interface with `put`, `get`, `forget`, `has` - an `InMemoryCache` implementation (backed by a private array) - a `FileCache` implementation (serializes to files on disk) - a `UserProfileService` class that depends on `Cache` to cache expensive `getProfile(int $id)` lookups

Show how you'd wire it up in `InMemoryCache` mode for tests vs `FileCache` in prod.

Q10.3 (Principle audit)

After you've written Q10.2, go back through your code and label — in a comment — which episode's principle each decision came from. For example:

```
// Ep 4: depend on the interface, not the concrete cache class
public function __construct(private Cache $cache) {}
```

Every constructor, every interface, every visibility modifier should trace back to a specific episode.

Q10.4 (Stretch)

Add a `LoggingCache` that **wraps** another `Cache` and logs every call before delegating to the inner one. This is the **Decorator pattern** — a pure composition play. What property of your existing interfaces makes this possible without changing any of them?

Final Synthesis Questions

These tie multiple episodes together.

QS.1

Walk through this sentence and name every principle it touches:

> "Inject a `readonly` DTO into a service that depends on a `Storage` interface rather than `S3Storage`, and use `private(set)` visibility on the DTO's id property."

QS.2

Your Laravel controller currently looks like this:

```
public function store(Request $request) {
    $user = \App\Models\User::create($request->all());
    \Mail::to($user->email)->send(new WelcomeMail());
    \Log::info("Registered {$user->email}");
    return redirect('/dashboard');
```

```
}
```

From the principles of this course, identify **three** things wrong with it and rewrite it.

QS.3

When is inheritance genuinely the right tool, not composition? Give one concrete example.

QS.4

If you had to pick **three** takeaways from the whole course to tattoo on your forearm, which would they be and why?

Ready to start? Say "quiz me on OOP Ep 1" and I'll walk you through Q1.1 → Q1.4 one at a time. Or if you want to tackle them on your own, grind through a section and paste your answers.